

CMATH_REFERENCE

Core Definition

`<cmath>` is the C++ wrapper for C's `<math.h>`: floating-point mathematics — trigonometry, exponentials, logarithms, power, rounding, classification, and (C++17) special functions. Names live in namespace `std` and are overloaded for `float`, `double` and `long double`; C++11 adds integral overloads that promote to `double`.

#CPP #CMATH #MATH #TRIGONOMETRY #FLOATING-POINT #NUMERIC #SQRT #POW

COMPLETE CMATH QUICK REFERENCE

FUNCTIONS — Target | Operation | Output

// =====			
// BASIC OPERATIONS			
// =====			
<code>std::abs(x)</code>	// any numeric	Absolute value	Overloaded for int and float types
<code>std::fabs(x)</code>	// float	Absolute value	Always floating-point
<code>std::fmod(x, y)</code>	// two floats	Floating remainder	Sign follows x; $x - \text{trunc}(x/y)*y$
<code>std::remainder(x, y)</code>	// two floats	IEEE remainder	Rounds quotient to NEAREST; result may be negative
<code>std::remquo(x, y, &quo)</code>	// + int*	Remainder + quotient	Stores low bits of quotient
<code>std::fma(x, y, z)</code>	// three	$x*y + z$, one rounding	More accurate and often faster than $x*y+z$
<code>std::fmax(x, y)</code>	// two floats	Larger value	Ignores NaN if the other is a number
<code>std::fmin(x, y)</code>	// two floats	Smaller value	Ignores NaN if the other is a number
<code>std::fdim(x, y)</code>	// two floats	Positive difference	$\max(x-y, 0)$
// =====			
// EXPONENTIAL & LOGARITHMIC			
// =====			
<code>std::exp(x)</code>	// float	e^x	
<code>std::exp2(x)</code>	// float	2^x	C++11
<code>std::expm1(x)</code>	// float	$e^x - 1$	C++11; accurate for tiny x
<code>std::log(x)</code>	// float	Natural log $\ln(x)$	Domain error if $x < 0$
<code>std::log10(x)</code>	// float	Base-10 log	
<code>std::log2(x)</code>	// float	Base-2 log	C++11
<code>std::log1p(x)</code>	// float	$\ln(1 + x)$	C++11; accurate for tiny x
<code>std::logb(x)</code>	// float	Exponent as a float	C++11
<code>std::ilogb(x)</code>	// float	Exponent as an int	C++11
// =====			
// POWER & ROOTS			
// =====			
<code>std::pow(base, exp)</code>	// two floats	base^{exp}	Slow; prefer $x*x$ for squares
<code>std::sqrt(x)</code>	// float	Square root	NaN if $x < 0$
<code>std::cbrt(x)</code>	// float	Cube root	C++11; works for negative x
<code>std::hypot(x, y)</code>	// two floats	$\sqrt{x^2+y^2}$	C++11; avoids overflow/underflow
<code>std::hypot(x, y, z)</code>	// three	$\sqrt{x^2+y^2+z^2}$	C++17
// =====			
// TRIGONOMETRIC (angles in RADIANS)			
// =====			
<code>std::sin(x)</code>	// radians	Sine	
<code>std::cos(x)</code>	// radians	Cosine	
<code>std::tan(x)</code>	// radians	Tangent	
<code>std::asin(x)</code>	// [-1, 1]	Arcsine	Returns $[-\pi/2, \pi/2]$; NaN outside domain
<code>std::acos(x)</code>	// [-1, 1]	Arccosine	Returns $[0, \pi]$; NaN outside domain
<code>std::atan(x)</code>	// float	Arctangent	Returns $(-\pi/2, \pi/2)$

```

std::atan2(y, x)           // y THEN x   | Arctangent of y/x   | Returns (- $\pi$ ,  $\pi$ ]; quadrant-correct

// =====
// HYPERBOLIC
// =====
std::sinh(x) / std::cosh(x) / std::tanh(x) // Hyperbolic sine/cosine/tangent
std::asinh(x) / std::acosh(x) / std::atanh(x) // Inverse hyperbolic (C++11)
// acosh domain:  $x \geq 1$ ; atanh domain:  $|x| < 1$ 

// =====
// ROUNDING & INTEGER EXTRACTION
// =====
std::ceil(x)               // float      | Round UP           | Returns float; ceil(-2.5) = -2
std::floor(x)              // float      | Round DOWN        | Returns float; floor(-2.5) = -3
std::trunc(x)              // float      | Toward zero       | C++11; trunc(-2.7) = -2
std::round(x)              // float      | Nearest, .5 away from 0 | C++11; round(-2.5) = -3
std::lround(x) / std::llround(x) // float      | Nearest  $\rightarrow$  long/long long | C++11
std::nearbyint(x)          // float      | Nearest, current mode | C++11; no FE_INEXACT raised
std::rint(x)              // float      | Nearest, current mode | C++11; may raise FE_INEXACT
std::lrint(x) / std::llrint(x) // float      | Nearest  $\rightarrow$  long/long long | C++11
std::modf(x, &intpart)     // float+ptr   | Split int/frac parts | Returns fractional part
std::frexp(x, &exp)        // float+int*  | Split mantissa/exp   | Returns m in [0.5,1);  $x = m \cdot 2^{\text{exp}}$ 
std::ldexp(m, exp)         // float+int   |  $m \cdot 2^{\text{exp}}$       | Inverse of frexp
std::scalbn(x, n)          // float+int   |  $x \cdot \text{FLT\_RADIX}^n$  | C++11; efficient
std::scalbln(x, n)         // float+long  |  $x \cdot \text{FLT\_RADIX}^n$  | C++11

// =====
// CLASSIFICATION & COMPARISON (C++11)
// =====
std::isnan(x)              // float      | Is Not-a-Number     | Returns bool
std::isinf(x)              // float      | Is infinite         | Returns bool
std::isfinite(x)           // float      | Is finite (not NaN/inf) | Returns bool
std::isnormal(x)           // float      | Normal (not 0/sub/NaN) | Returns bool
std::signbit(x)            // float      | Sign bit set        | Returns bool; TRUE for -0.0
std::fpclassify(x)         // float      | Category            |
// FP_ZERO/FP_SUBNORMAL/FP_NORMAL/FP_INFINITY/FP_NAN
std::isgreater(x, y)       // two floats  |  $x > y$ , no FP exception | Returns bool
std::isgreaterequal(x, y)  // two floats  |  $x \geq y$                 | Returns bool
std::isless(x, y)          // two floats  |  $x < y$                   | Returns bool
std::islessequal(x, y)     // two floats  |  $x \leq y$                 | Returns bool
std::islessgreater(x, y)   // two floats  |  $x < y \ || \ x > y$       | Returns bool
std::isunordered(x, y)     // two floats  | Either is NaN       | Returns bool

// =====
// SIGN & NEXT-VALUE MANIPULATION
// =====
std::copysign(mag, sgn)     // two floats  | |mag| with sgn's sign | C++11
std::nextafter(from, to)    // two floats  | Next representable    | C++11; one ULP step toward `to`
std::nexttoward(from, to)   // + long double | Next representable    | C++11
std::nan("")               // string      | Quiet NaN             | Also nanf, nanl

// =====
// ERROR & GAMMA FUNCTIONS (C++11)
// =====
std::erf(x)                // float      | Error function        |
std::erfc(x)               // float      | Complementary erf     |  $1 - \text{erf}(x)$ , accurate for large x
std::tgamma(x)              // float      | True gamma  $\Gamma(x)$    | tgamma(n+1) == n!
std::lgamma(x)              // float      |  $\ln|\Gamma(x)|$          | Avoids overflow for large x

// =====
// LINEAR INTERPOLATION (C++20, in <cmath>)
// =====
std::lerp(a, b, t)          // three      |  $a + t \cdot (b - a)$     | C++20; exact at  $t=0$  and  $t=1$ ,
// monotonic

// =====
// SPECIAL MATHEMATICAL FUNCTIONS (C++17)
// =====
std::assoc_laguerre(n, m, x) // Associated Laguerre polynomials
std::assoc_legendre(l, m, x) // Associated Legendre polynomials
std::beta(x, y)             // Beta function B(x,y)
std::comp_ellint_1(k)        // Complete elliptic integral, 1st kind

```

```

std::comp_ellint_2(k)           // Complete elliptic integral, 2nd kind
std::comp_ellint_3(k, nu)      // Complete elliptic integral, 3rd kind
std::ellint_1(k, phi)          // Incomplete elliptic integral, 1st kind
std::ellint_2(k, phi)          // Incomplete elliptic integral, 2nd kind
std::ellint_3(k, nu, phi)      // Incomplete elliptic integral, 3rd kind
std::cyl_bessel_i(nu, x)       // Modified cylindrical Bessel, 1st kind
std::cyl_bessel_j(nu, x)       // Cylindrical Bessel, 1st kind
std::cyl_bessel_k(nu, x)       // Modified cylindrical Bessel, 2nd kind
std::cyl_neumann(nu, x)        // Cylindrical Neumann (Bessel 2nd kind)
std::sph_bessel(n, x)          // Spherical Bessel, 1st kind
std::sph_neumann(n, x)         // Spherical Neumann
std::sph_legendre(l, m, theta) // Spherical associated Legendre
std::expint(x)                 // Exponential integral Ei(x)
std::hermite(n, x)             // Hermite polynomials
std::laguerre(n, x)            // Laguerre polynomials
std::legendre(l, x)            // Legendre polynomials
std::riemann_zeta(x)           // Riemann zeta function
// Each has f/l suffixed variants (e.g. betaf, betal). libstdc++ supports these;
// libc++ support is incomplete – check before relying on them.

// =====
// MACROS & CONSTANTS
// =====
HUGE_VAL / HUGE_VALF / HUGE_VALL // Overflow return value (usually infinity)
INFINITY                          // Positive infinity (float)
NaN                               // Quiet NaN (float)
MATH_ERRNO / MATH_ERREXCEPT    // Error-reporting mechanism bits
math_errhandling                 // Which mechanisms are active
FP_ILOGB0 / FP_ILOGBNAN          // ilogb return for 0 / NaN
FP_FAST_FMA / FP_FAST_FMAF      // Defined if fma is hardware-fast

```

MATHEMATICAL CONSTANTS — <numbers> (C++20)

```

#include <numbers>
std::numbers::e           // 2.718281828459045 Euler's number
std::numbers::log2e        // 1.442695040888963 log2(e)
std::numbers::log10e       // 0.434294481903252 log10(e)
std::numbers::pi           // 3.141592653589793 π
std::numbers::inv_pi       // 0.318309886183791 1/π
std::numbers::inv_sqrtpi   // 0.564189583547756 1/√π
std::numbers::ln2          // 0.693147180559945 ln(2)
std::numbers::ln10         // 2.302585092994046 ln(10)
std::numbers::sqrt2        // 1.414213562373095 √2
std::numbers::sqrt3        // 1.732050807568877 √3
std::numbers::inv_sqrt3    // 0.577350269189626 1/√3
std::numbers::egamma       // 0.577215664901533 Euler-Mascheroni γ
std::numbers::phi          // 1.618033988749895 Golden ratio φ
// Suffixed variants: pi_v<float>, pi_v<long double>, etc.

// Pre-C++20 POSIX macros (NOT standard C++, may need _USE_MATH_DEFINES on MSVC):
// M_PI, M_E, M_SQRT2, M_LN2, M_PI_2, M_PI_4, M_1_PI, M_2_PI
// Portable fallback: constexpr double PI = 3.14159265358979323846;

```

NUMERIC LIMITS — <limits>

```

#include <limits>
std::numeric_limits<double>::max()           // Largest finite value
std::numeric_limits<double>::min()           // Smallest POSITIVE normal value (not the most negative!)
std::numeric_limits<double>::lowest()        // Most negative finite value (C++11)
std::numeric_limits<double>::epsilon()       // Machine epsilon: 1.0 → next representable gap
std::numeric_limits<double>::infinity()      // +∞
std::numeric_limits<double>::quiet_NaN()     // NaN
std::numeric_limits<double>::denorm_min()    // Smallest subnormal
std::numeric_limits<double>::digits10       // Guaranteed decimal digits (15 for double)
std::numeric_limits<double>::max_digits10   // Digits for exact round-trip (17 for double)
std::numeric_limits<double>::is_iec559      // True if IEEE 754 compliant

```

COMMON PATTERNS & EXAMPLES

Degrees and Radians

```
#include <cmath>
#include <numbers>

constexpr double deg2rad(double d) { return d * std::numbers::pi / 180.0; }
constexpr double rad2deg(double r) { return r * 180.0 / std::numbers::pi; }

double y = std::sin(deg2rad(30.0));    // 0.5
```

Comparing Floating-Point Values

```
#include <cmath>
#include <limits>
#include <algorithm>

// Never write: if (a == b) for computed doubles.

bool nearlyEqual(double a, double b,
                 double relTol = 1e-9,
                 double absTol = 1e-12) {
    if (a == b) return true;           // Handles infinities
    double diff = std::fabs(a - b);
    if (diff <= absTol) return true;   // Near zero
    return diff <= relTol * std::max(std::fabs(a), std::fabs(b));
}

// ULP-based comparison for values known to be close:
bool withinUlp(double a, double b, int ulps = 4) {
    for (int i = 0; i < ulps; ++i) {
        if (a == b) return true;
        a = std::nextafter(a, b);
    }
    return a == b;
}
```

Detecting and Handling Bad Values

```
double x = compute();

if (std::isnan(x))      std::cerr << "NaN produced\n";
else if (std::isinf(x)) std::cerr << (x > 0 ? "+inf\n" : "-inf\n");
else if (!std::isnormal(x) && x != 0) std::cerr << "subnormal\n";

// NaN != NaN - this is the one place == behaves unexpectedly by design:
double n = std::numeric_limits<double>::quiet_NaN();
bool b = (n == n);      // false
bool c = std::isnan(n); // true - always test this way
```

Rounding Behavior

```
double vals[] = {2.5, -2.5, 2.4, -2.4};

// x      ceil  floor  trunc  round  nearbyint(default banker's)
// 2.5     3     2     2     3     2
// -2.5    -2    -3    -2    -3    -2
// 2.4     3     2     2     2     2
// -2.4    -2    -3    -2    -2    -2

// Rounding to N decimal places (careful – not exact in binary FP):
double round2(double v) { return std::round(v * 100.0) / 100.0; }
// For display, prefer formatting: std::format("{:.2f}", v)
```

Integer Division Pitfalls

```
int a = 7, b = 2;
double bad = a / b;           // 3.0 – integer division happened FIRST
double good = static_cast<double>(a) / b; // 3.5

// Ceiling division on integers, no floating point needed:
int ceilDiv(int n, int d) { return (n + d - 1) / d; } // For positive n, d
```

Safe `sqrt` and `log`

```
double safeSqrt(double x) {
    if (x < 0) throw std::domain_error("sqrt of negative");
    return std::sqrt(x);
}

// Guard against log(0) → -inf and log(negative) → NaN
double safeLog(double x) {
    if (x <= 0) return -std::numeric_limits<double>::infinity();
    return std::log(x);
}
```

`pow` Is Slower Than You Think

```
double x = 3.7;

double sq  = x * x;           // Fast
double cube = x * x * x;      // Fast
double p   = std::pow(x, 2.0); // Much slower – general algorithm
double r   = std::sqrt(x);     // Faster than pow(x, 0.5)

// pow with integer exponents can also lose precision:
// pow(10.0, 2.0) may give 99.99999999999986 on some implementations.
```

Distance and Magnitude

```
// Naive – can overflow when components are large:
double d = std::sqrt(dx*dx + dy*dy);

// Robust:
double d2 = std::hypot(dx, dy); // C++11
double d3 = std::hypot(dx, dy, dz); // C++17, 3D

// When only comparing distances, skip the sqrt entirely:
double distSq = dx*dx + dy*dy; // Compare squared distances
```

Angles from Coordinates

```
// atan2 takes y FIRST and handles all four quadrants + x == 0:
double angle = std::atan2(y, x);           // (-π, π]

// atan(y/x) loses the quadrant and divides by zero at x == 0.
```

Splitting a Number

```
double intPart;
double fracPart = std::modf(3.75, &intPart); // intPart = 3.0, fracPart = 0.75

int exp;
double mant = std::frexp(8.0, &exp);         // mant = 0.5, exp = 4  (0.5 · 24 = 8)
double back = std::ldexp(mant, exp);         // 8.0
```

Accumulating Without Drift

```
// Naive accumulation of many small values loses precision:
double sum = 0;
for (double v : values) sum += v;

// Kahan summation – compensates for the lost low-order bits:
double kahanSum(const std::vector<double>& values) {
    double sum = 0.0, c = 0.0;
    for (double v : values) {
        double y = v - c;
        double t = sum + y;
        c = (t - sum) - y;
        sum = t;
    }
    return sum;
}

// std::fma also helps in dot products – one rounding instead of two:
double dot = 0;
for (std::size_t i = 0; i < n; ++i) dot = std::fma(a[i], b[i], dot);
```

Factorials and Combinatorics

```
// tgamma(n+1) == n! – works for larger n than an integer factorial
double factorial(int n) { return std::tgamma(n + 1.0); }

// For large n, work in log space to avoid overflow:
double logBinomial(int n, int k) {
    return std::lgamma(n + 1.0) - std::lgamma(k + 1.0) - std::lgamma(n - k + 1.0);
}
```

Clamping and Interpolation

```
#include <algorithm>
#include <cmath>

double c = std::clamp(x, 0.0, 1.0);           // <algorithm>, C++17
double v = std::lerp(a, b, t);               // <cmath>, C++20 – safer than a + t*(b-a)

// Smoothstep
double smoothstep(double e0, double e1, double x) {
    double t = std::clamp((x - e0) / (e1 - e0), 0.0, 1.0);
    return t * t * (3.0 - 2.0 * t);
}
```

Wrapping Values

```
// fmod keeps the sign of x – often not what you want for a wrap:
double wrapped = std::fmod(-1.0, 360.0);           // -1.0

// Positive-result modulo:
double posMod(double x, double m) {
    double r = std::fmod(x, m);
    return r < 0 ? r + m : r;                      // 359.0
}
```

Checking for Errors

```
#include <cmath>
#include <cfenv>
#include <cerrno>

std::feclearexcept(FE_ALL_EXCEPT);
errno = 0;

double r = std::log(-1.0);

if (errno == EDOM)                std::cerr << "domain error\n";
if (std::fetestexcept(FE_INVALID)) std::cerr << "FE_INVALID raised\n";
if (std::fetestexcept(FE_DIVBYZERO)) std::cerr << "division by zero\n";
if (std::fetestexcept(FE_OVERFLOW)) std::cerr << "overflow\n";
// Requires: #pragma STDC FENV_ACCESS ON (support varies by compiler)
```

IMPORTANT CONCEPTS

Everything Is in Radians

`sin`, `cos`, `tan` and their inverses all use radians. Converting from degrees is the caller's job — a missing $\pi/180$ is the most common numerical bug in graphics and geometry code.

`atan2(y, x)` — Argument Order and Why It Exists

The y-coordinate comes first. Unlike `atan(y/x)`, it knows which quadrant the point is in and behaves correctly when `x == 0`. Use it for any angle-from-vector computation.

`abs` vs `fabs` — the `<cstdlib>` Trap

```
#include <cmath>
std::abs(-3.7);           // 3.7 – correct, <cmath> overload

#include <cstdlib>
abs(-3.7);                // 3 (!) – the C int version, silently truncates
```

Always write `std::abs` with `<cmath>` included, or `std::fabs` to be unambiguous. Never use the unqualified global `abs` on floating-point values.

`numeric_limits<T>::min()` Is Not the Minimum

For floating-point types, `min()` is the smallest positive *normal* value (about $2.2\text{e-}308$ for `double`). The most negative value is `lowest()` (C++11). For integer types, `min()` does mean the most negative.

NaN Propagates and Never Compares Equal

Any arithmetic involving NaN yields NaN, and every comparison against NaN (including `==` with itself) is false. This silently makes sorts and containers misbehave — validate inputs at the boundary rather than tracking NaN through a computation.

Signed Zero Exists

`-0.0 == 0.0` is true, but `std::signbit(-0.0)` is true and `1.0 / -0.0` is `-inf`. `std::copysign` and `signbit` are the way to inspect it.

float VS double Precision

`float` gives roughly 7 significant decimal digits, `double` roughly 15-16. Use `double` unless memory or SIMD width forces otherwise; the speed difference on modern hardware is usually negligible for scalar code.

Integral Arguments Promote (C++11)

`std::sqrt(4)` compiles and returns `2.0` — C++11 added `double` overloads for integral arguments. Pre-C++11 that was a compile error, and code written then often has unnecessary casts.

fmod VS remainder

`fmod(x, y)` truncates the quotient toward zero, so the result carries `x`'s sign. `remainder(x, y)` rounds the quotient to nearest, so the result lies in `[-|y|/2, |y|/2]` and may have the opposite sign. Choose deliberately.

fma Is One Rounding, Not Two

`std::fma(a, b, c)` computes `a*b + c` with a single rounding at the end. It is more accurate than the separate expression, and on hardware with an FMA instruction (`FP_FAST_FMA` defined) it is also faster.

Compiler Fast-Math Changes Semantics

`-ffast-math` / `/fp:fast` lets the compiler assume no NaN or infinity, reorder operations, and skip signed-zero handling. It can silently break `isnan` checks and Kahan summation. Do not enable it in code that relies on IEEE semantics.

errno vs Floating-Point Exceptions

Which mechanism reports math errors is implementation-defined and reported by `math_errhandling`. Checking `errno` requires clearing it first; checking FP exception flags requires `<cfenv>` and `FENV_ACCESS`. In practice, validating inputs beforehand is more reliable than either.

DOMAIN & RANGE QUICK TABLE

Function	Valid domain	Result outside domain
<code>sqrt(x)</code>	<code>x >= 0</code>	NaN
<code>log(x)</code>	<code>x > 0</code>	<code>x==0</code> → <code>-inf</code> ; <code>x<0</code> → NaN
<code>log10 / log2</code>	<code>x > 0</code>	same as log
<code>asin / acos</code>	<code>-1 <= x <= 1</code>	NaN
<code>acosh(x)</code>	<code>x >= 1</code>	NaN
<code>atanh(x)</code>	<code>-1 < x < 1</code>	<code> x ==1</code> → <code>±inf</code> ; <code> x >1</code> → NaN
<code>pow(x, y)</code>	<code>x>=0</code> , or <code>y</code> integral	negative base + fractional exp → NaN
<code>tgamma(x)</code>	<code>x</code> not a non-positive integer	<code>±inf</code> or NaN
<code>fmod(x, 0)</code>	<code>y != 0</code>	NaN

BEST PRACTICES

1. **Never compare floats with `==`** — use a relative + absolute tolerance
2. **Include `<cmath>` and qualify with `std::`** — the unqualified `abs` may be the int version
3. **Convert degrees to radians explicitly**; write a named helper
4. **Use `atan2(y, x)`**, not `atan(y/x)`, for angles
5. **Prefer `x*x` over `pow(x, 2)`** — faster and exact
6. **Use `hypot` for magnitudes** to avoid intermediate overflow
7. **Use `std::numbers::pi` (C++20)** instead of `M_PI` or a hand-typed literal
8. **Validate domains before calling `sqrt`, `log`, `asin`, `acos`**
9. **Check `isnan`/`isfinite`** at boundaries where bad values enter
10. **Use `lowest()`**, not `min()`, for the most negative floating-point value
11. **Use `fma` in accumulation loops** for accuracy and speed
12. **Prefer Kahan summation** when adding many values of varying magnitude
13. **Avoid `-ffast-math`** in numerically sensitive code
14. **Compare squared distances** when you only need an ordering
15. **Use `std::lerp` and `std::clamp`** rather than hand-rolled equivalents

RELATED HEADERS

```
#include <cmath>           // The math functions
#include <numbers>          // C++20: pi, e, sqrt2, phi ...
#include <limits>           // numeric limits: epsilon, infinity, max, lowest
#include <cfenv>            // Floating-point environment: rounding mode, exception flags
#include <cstdlib>          // Integer abs, div, rand
#include <complex>         // Complex arithmetic and its own transcendental functions
#include <valarray>         // Element-wise array math
#include <numeric>          // accumulate, inner_product, gcd, lcm, midpoint, iota
#include <random>           // Proper random number generation (not rand())
#include <algorithm>        // clamp, min, max, minmax
#include <bit>              // C++20: bit_cast, popcount, bit_width
#include <ratio>            // Compile-time rational arithmetic
```

EXTERNAL RESOURCES

- **Common mathematical functions:** <https://en.cppreference.com/w/cpp/numeric/math>
- **`<cmath>`:** <https://en.cppreference.com/w/cpp/header/cmath>
- **Mathematical special functions:** https://en.cppreference.com/w/cpp/numeric/special_functions
- **`<numbers>`:** <https://en.cppreference.com/w/cpp/numeric/constants>
- **What Every Computer Scientist Should Know About Floating-Point:** https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

Standard: C++11 (classification, `cbrt`, `hypot`, `round`) / C++17 (special functions, 3-arg `hypot`) / C++20 (`lerp`, `<numbers>`) **Last Updated:** September 2026